

C Problems and Solutions

atoi impl solution

```
/* atoi_solution.c */
#include <stdio.h>
#include <ctype.h>

int my_atoi(const char *s) {
    while (isspace(*s)) s++;
    int sign = 1; if (*s=='+'||*s=='-') { if (*s=='-') sign=-1; s++; }
    int val = 0;
    while (*s && isdigit(*s)) { val = val*10 + (*s - '0'); s++; }
    return sign*val;
}

int main(){ printf("%d\n", my_atoi(" -123 ")); return 0; }
```

atoi impl starter

```
/* atoi_starter.c */
#include <stdio.h>
#include <ctype.h>
int my_atoi(const char *s) { /* TODO */ }
int main(){ printf("%d\n", my_atoi(" -123 ")); return 0; }
```

binary search solution

```
/* bsearch_solution.c */
#include <stdio.h>
int bsearch_idx(int *a, int n, int key) {
    int lo=0, hi=n-1;
    while (lo<=hi) {
        int mid = lo + (hi-lo)/2;
        if (a[mid]==key) return mid;
        else if (a[mid]<key) lo=mid+1;
        else hi=mid-1;
    }
    return -1;
}
int main(){ int a[]={1,3,5,7,9}; printf("%d\n", bsearch_idx(a,5,7)); return 0; }
```

binary search starter

```
/* bsearch_starter.c */
#include <stdio.h>
int bsearch_idx(int *a, int n, int key) { /* TODO */ }
int main(){ int a[]={1,3,5,7,9}; printf("%d\n", bsearch_idx(a,5,7)); return 0; }
```

bit count solution

```
/* bit_count_solution.c */
#include <stdio.h>
unsigned int popcount(unsigned int x) {
    unsigned int cnt = 0;
    while (x) { x &= x-1; cnt++; }
    return cnt;
}
```

```

}
int main() { printf("%u\n", popcount(0b101101)); return 0; }

```

bit count starter

```

/* bit_count_starter.c */
#include <stdio.h>
unsigned int popcount(unsigned int x) {
    // TODO: implement (Brian Kernighan)
}
int main() { printf("%u\n", popcount(0b101101)); return 0; }

```

linked list solution

```

/* linked_list_solution.c */
#include <stdio.h>
#include <stdlib.h>

typedef struct node { int val; struct node *next; } node;

node* insert_front(node* head, int v) {
    node* n = malloc(sizeof(node));
    n->val = v; n->next = head; return n;
}
node* delete_value(node* head, int v) {
    node dummy; dummy.next = head;
    node *prev = &dummy, *cur = head;
    while (cur) {
        if (cur->val == v) { prev->next = cur->next; free(cur); break; }
        prev = cur; cur = cur->next;
    }
    return dummy.next;
}
void print_list(node* head) {
    while (head) { printf("%d -> ", head->val); head = head->next; } printf("NULL\n");
}

int main() {
    node* h = NULL;
    h = insert_front(h, 3);
    h = insert_front(h, 5);
    h = insert_front(h, 7);
    print_list(h);
    h = delete_value(h, 5);
    print_list(h);
    return 0;
}

```

linked list starter

```

/* linked_list_starter.c */
#include <stdio.h>
typedef struct node { int val; struct node *next; } node;
// Implement insert_front, delete_value, print_list

```

matrix mul solution

```

/* matrix_mul_solution.c */
#include <stdio.h>
#define N 3

void matmul(int A[N][N], int B[N][N], int C[N][N]) {

```

```

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                C[i][j] = 0;
                for (int k = 0; k < N; k++) C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

int main() {
    int A[N][N] = {{1,2,3},{4,5,6},{7,8,9}};
    int B[N][N] = {{9,8,7},{6,5,4},{3,2,1}};
    int C[N][N] = {0};
    matmul(A,B,C);
    for (int i=0;i<N;i++){ for(int j=0;j<N;j++) printf("%d ", C[i][j]); printf("\n"); }
    return 0;
}

```

matrix mul starter

```

/* matrix_mul_starter.c */
#include <stdio.h>
#define N 3

void matmul(int A[N][N], int B[N][N], int C[N][N]) {
    // TODO
}

int main() {
    int A[N][N] = {{1,2,3},{4,5,6},{7,8,9}};
    int B[N][N] = {{9,8,7},{6,5,4},{3,2,1}};
    int C[N][N] = {0};
    matmul(A,B,C);
    for (int i=0;i<N;i++){ for(int j=0;j<N;j++) printf("%d ", C[i][j]); printf("\n"); }
    return 0;
}

```

max in array solution

```

/* max_in_array_solution.c */
#include <stdio.h>

int find_max(int *arr, int n) {
    if (n <= 0) return 0;
    int m = arr[0];
    for (int i = 1; i < n; i++) if (arr[i] > m) m = arr[i];
    return m;
}

int main() {
    int a[] = {3, 1, 4, 1, 5, 9};
    printf("%d\n", find_max(a, 6));
    return 0;
}

```

max in array starter

```

/* max_in_array_starter.c */
#include <stdio.h>

int find_max(int *arr, int n) {
    // TODO
}

int main() {

```

```

    int a[] = {3, 1, 4, 1, 5, 9};
    printf("%d\n", find_max(a, 6));
    return 0;
}

```

memmove impl solution

```

/* memmove_solution.c */
#include <stdio.h>
#include <string.h>

void *my_memmove(void *dst, const void *src, size_t n) {
    unsigned char *d = dst; const unsigned char *s = src;
    if (d < s) {
        for (size_t i=0; i<n; i++) d[i]=s[i];
    } else if (d > s) {
        for (size_t i=n; i>0; i--) d[i-1]=s[i-1];
    }
    return dst;
}

int main(){ char b[20] = "HelloWorld"; my_memmove(b+2,b,5); printf("%s\n", b); return 0; }

```

memmove impl starter

```

/* memmove_starter.c */
#include <stdio.h>
#include <string.h>
void *my_memmove(void *dst, const void *src, size_t n) { /* TODO */ }
int main(){ char b[20] = "HelloWorld"; my_memmove(b+2,b,5); printf("%s\n", b); return 0; }

```

queue stacks solution

```

/* queue_stacks_solution.c */
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
typedef struct { int a[MAX]; int top; } stack;
void spush(stack *s, int v){ s->a[s->top++]=v; }
int spop(stack *s){ return s->a[--s->top]; }

typedef struct { stack s1, s2; } queue_ts;
void enqueue(queue_ts *q, int v){ spush(&q->s1, v); }
int dequeue(queue_ts *q){
    if (q->s2.top==0) while (q->s1.top>0) spush(&q->s2, spop(&q->s1));
    return spop(&q->s2);
}

int main() {
    queue_ts q = {{.top=0},{.top=0}};
    enqueue(&q,1); enqueue(&q,2); enqueue(&q,3);
    printf("%d\n", dequeue(&q));
    enqueue(&q,4);
    printf("%d\n", dequeue(&q));
    return 0;
}

```

queue stacks starter

```

/* queue_stacks_starter.c */
#include <stdio.h>
// Implement queue using two stacks (arrays)

```

reverse string solution

```
/* reverse_string_solution.c */
#include <stdio.h>
#include <string.h>

void reverse(char *str) {
    int n = strlen(str);
    for (int i = 0; i < n/2; i++) {
        char tmp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = tmp;
    }
}

int main() {
    char s[] = "HELLO";
    reverse(s);
    printf("%s\n", s);
    return 0;
}
```

reverse string starter

```
/* reverse_string_starter.c */
#include <stdio.h>
#include <string.h>

void reverse(char *str) {
    // TODO: implement
}

int main() {
    char s[] = "HELLO";
    reverse(s);
    printf("%s\n", s);
    return 0;
}
```

stack array solution

```
/* stack_array_solution.c */
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
typedef struct { int a[MAX]; int top; } stack;

void push(stack *s, int v) { if (s->top >= MAX) { printf("overflow\n"); exit(1);} s->a[s->top++] = v; }
int pop(stack *s) { if (s->top <= 0) { printf("underflow\n"); exit(1);} return s->a[--s->top]; }
int peek(stack *s) { if (s->top <= 0) { printf("empty\n"); return -1;} return s->a[s->top-1]; }

int main() {
    stack s = {.top = 0};
    push(&s, 1); push(&s, 2); push(&s, 3);
    printf("%d\n", pop(&s));
    printf("%d\n", peek(&s));
    return 0;
}
```

stack array starter

```
/* stack_array_starter.c */  
#include <stdio.h>  
#define MAX 10  
typedef struct { int a[MAX]; int top; } stack;  
// implement push pop peek
```